

Week 12 – Interview preparation – Angular Track

1. What are the main features of Chrome DevTools useful for Angular debugging?

Answer:

Chrome DevTools provides features like:

- Elements panel (DOM inspection)
 - Console (logging & error tracking)
 - Sources (debugging with breakpoints)
 - Network (HTTP request tracking)
 - Performance (profiling)
 - Application tab (storage, service workers)
-

2. How do you set breakpoints in Angular using DevTools?

Answer:

Go to the **Sources** tab, locate the TypeScript/JS file, and click on the line number to set a breakpoint. When the code executes, it pauses at that line.

3. What is the purpose of source maps in Angular debugging?

Answer:

Source maps map compiled JavaScript back to original TypeScript code, allowing developers to debug in the original source instead of minified/transpiled JS.

4. How can you inspect component state in Chrome DevTools?

Answer:

Using the **Console** or Angular DevTools extension, you can access component instances and inspect properties, state, and bindings.

5. What is Angular DevTools and how does it help debugging?

Answer:

Angular DevTools is a browser extension that provides component tree visualization, state inspection, and performance profiling for Angular apps.

6. How do you debug HTTP requests in Angular?

Answer:

Use the **Network tab** in DevTools to monitor API calls, inspect headers, payloads, responses, and timing.

7. What is the use of the Console tab in debugging Angular apps?

Answer:

It helps log messages, errors, warnings, and allows execution of JavaScript commands for runtime inspection.

8. How can you detect performance issues in Angular apps?

Answer:

Use the **Performance tab** in DevTools to record runtime behavior, analyze change detection cycles, and identify bottlenecks.

9. What are conditional breakpoints?

Answer:

Breakpoints that pause execution only when a specified condition evaluates to true.

10. How do you debug event listeners in Angular?

Answer:

Use the **Event Listener Breakpoints** section in DevTools to pause execution when specific DOM events are triggered.

11. How do you configure debugging for Angular in VS Code?

Answer:

By creating a launch.json file with a Chrome debugger configuration pointing to `http://localhost:4200`.

12. What extension is required for debugging Angular in VS Code?

Answer:

Debugger for Chrome (or built-in JavaScript debugger in modern VS Code versions).

13. How do you set breakpoints in VS Code?

Answer:

Click in the gutter next to the line number in a TypeScript file to set breakpoints.

14. What is the role of launch.json in debugging?

Answer:

It defines debug configurations like URL, browser type, source maps, and file paths.

15. How do you debug TypeScript directly in VS Code?

Answer:

Using source maps, VS Code maps the running JS to TS files, allowing direct debugging in TypeScript.

16. What are watch expressions in VS Code debugging?

Answer:

They allow you to monitor variable values or expressions during debugging sessions.

17. How can you inspect variables during debugging?

Answer:

Using the **Variables panel**, hover tooltips, or the Debug Console.

18. What is step over, step into, and step out?

Answer:

- Step Over: Executes next line without entering functions
 - Step Into: Goes inside function calls
 - Step Out: Exits current function
-

19. How do you debug asynchronous code in Angular?

Answer:

Use breakpoints and async call stacks in VS Code or DevTools to trace promises, observables, and callbacks.

20. What are common issues faced while debugging Angular apps?

Answer:

- Source maps not working
- Incorrect path mappings
- Production build minification

- Async code complexity
- Change detection issues

21. What is Agile methodology?

Answer:

Agile is an iterative approach to software development that emphasizes collaboration, customer feedback, and small, rapid releases.

22. What are the key values of the Agile Manifesto?

Answer:

1. Individuals and interactions over processes and tools
 2. Working software over comprehensive documentation
 3. Customer collaboration over contract negotiation
 4. Responding to change over following a plan
-

23. Name the 12 principles of Agile.

Answer:

1. Customer satisfaction via early delivery
 2. Welcome changing requirements
 3. Frequent delivery of working software
 4. Close collaboration between business & developers
 5. Motivated individuals
 6. Face-to-face conversation
 7. Working software is the primary measure
 8. Sustainable development pace
 9. Technical excellence & good design
 10. Simplicity
 11. Self-organizing teams
 12. Regular reflection & improvement
-

24. How does Agile differ from Waterfall in handling requirements?

Answer:

Agile embraces changing requirements at any stage; Waterfall defines requirements upfront and changes are costly.

25. Why is customer collaboration emphasized in Agile?**Answer:**

It ensures the product meets real user needs and allows feedback-driven changes early in development.

26. What are common Agile frameworks?**Answer:**

Scrum, Kanban, Lean, Extreme Programming (XP), SAFe.

27. What is an Agile iteration or sprint?**Answer:**

A time-boxed period (usually 1–4 weeks) during which a team delivers a potentially shippable increment of software.

28. How does Agile handle risk management?**Answer:**

Through iterative development, frequent reviews, and incremental delivery, Agile identifies and mitigates risks early.

29. What are the roles in Scrum?**Answer:**

- **Product Owner:** Defines backlog and priorities
 - **Scrum Master:** Facilitates Scrum process, removes impediments
 - **Development Team:** Cross-functional members who build the product
-

30. What are the main Scrum artifacts?**Answer:**

- Product Backlog
- Sprint Backlog
- Increment (shippable product)

31. What are Scrum ceremonies/events?

Answer:

- Sprint Planning
- Daily Stand-up
- Sprint Review
- Sprint Retrospective

32. What is a Sprint Goal?

Answer:

A short description of what the team aims to achieve during a sprint, providing focus and purpose.

33. How is the Product Backlog different from the Sprint Backlog?

Answer:

Product Backlog: All desired features prioritized by value.

Sprint Backlog: Items from the Product Backlog selected for the current sprint with a plan for delivery.

34. What is the Definition of Done (DoD)?

Answer:

A shared understanding of when a backlog item or increment is considered complete.

35. What is a burn-down chart?

Answer:

A graphical representation of remaining work versus time, showing progress toward sprint completion.

36. What is the purpose of a Sprint Retrospective?

Answer:

To reflect on the sprint and identify improvements in processes, collaboration, and quality.

37. How does Scrum handle changing requirements mid-sprint?

Answer:

Requirements changes are generally deferred to the next sprint; mid-sprint changes require negotiation with the Product Owner.

38. How do you measure Scrum team productivity?**Answer:**

Common metrics include velocity, sprint burndown, cycle time, and lead time.

39. What is Agile estimation?**Answer:**

Estimating the effort required to complete backlog items using relative sizing techniques like story points or t-shirt sizes.

40. What are story points?**Answer:**

A unit of measure for estimating the relative effort or complexity of a user story, not tied to hours.

41. What is Planning Poker?**Answer:**

A consensus-based technique for estimating effort using cards representing story points to reach agreement within a team.

42. What is velocity in Agile?**Answer:**

The number of story points or work items a team completes in a sprint; used for forecasting future sprints.

43. What is release planning?**Answer:**

A high-level plan outlining features to be delivered in upcoming releases, based on team velocity and priorities.

44. What is capacity planning in Agile?

Answer:

Estimating the amount of work a team can realistically complete in a sprint considering availability and team size.

45. How do Agile teams handle estimation uncertainty?**Answer:**

By using relative estimation, breaking down stories into smaller tasks, and continuously adjusting based on actual velocity.

46. What is backlog grooming/refinement?**Answer:**

An ongoing process to review, clarify, and prioritize backlog items for future sprints.

47. What is a user story?**Answer:**

A short, simple description of a feature from an end-user perspective: "As a [user], I want [feature] so that [benefit]."

48. What is the INVEST acronym for good user stories?**Answer:**

- **I:** Independent
 - **N:** Negotiable
 - **V:** Valuable
 - **E:** Estimable
 - **S:** Small
 - **T:** Testable
-

49. What is the difference between a user story and a use case?**Answer:**

User stories are short, simple, and focused on value; use cases are detailed functional flows covering multiple steps and actors.

50. How are acceptance criteria used in Agile user stories?

Answer:

They define the conditions that must be met for a story to be considered done, providing a clear testable definition.

51. What are common Agile metrics?

Answer:

Velocity, burn-down/burn-up charts, cycle time, lead time, cumulative flow diagrams, and defect density.

52. What is the difference between burn-down and burn-up charts?

Answer:

Burn-down: Shows remaining work over time.

Burn-up: Shows completed work over time, often alongside total scope.

53. What is a cumulative flow diagram?

Answer:

A visual representation of work items in different workflow states, helping identify bottlenecks.

54. How do you measure team predictability?

Answer:

By tracking velocity consistency, completed vs. planned work, and on-time delivery of sprint goals.

55. What is the purpose of Agile reporting?

Answer:

To provide visibility, track progress, support decision-making, and facilitate continuous improvement.

56. What is SDLC?

Answer:

Software Development Life Cycle: a structured process for developing software, typically includes phases like requirement analysis, design, development, testing, deployment, and maintenance.

57. How does SDLC differ from Agile?

Answer:

SDLC (like Waterfall) is linear and sequential, with fixed requirements upfront. Agile is iterative, flexible, and adaptive to change.

58. What are the disadvantages of SDLC compared to Agile?

Answer:

- Harder to accommodate changes
 - Delayed customer feedback
 - Higher risk of project failure if requirements were wrong
 - Late delivery of working software
-

59. Can Agile and SDLC coexist?

Answer:

Yes, Agile can be applied within SDLC frameworks as iterative increments, often called hybrid models.

60. How does Agile reduce risk compared to traditional SDLC?

Answer:

By delivering incremental working software, getting early feedback, allowing requirement changes, and promoting collaboration.

61. What is Git and why is it used?

Answer:

Git is a distributed version control system used to track changes in code, collaborate with multiple developers, and manage project history efficiently.

62. What is the difference between Git and GitHub?

Answer:

- **Git:** A version control system for tracking local code changes.
 - **GitHub:** A cloud-based platform to host Git repositories and facilitate collaboration.
-

63. What is git rebase and how does it differ from git merge?

Answer:

- **Rebase:** Moves or reapplies commits from one branch onto another, creating a linear history.
 - **Merge:** Combines two branches and preserves branch history, creating a merge commit.
-

64. What is the difference between git pull and git fetch?

Answer:

- git fetch: Downloads commits from a remote branch but doesn't merge.
 - git pull: Fetches commits and merges them into the current branch.
-

65. What is a fast-forward merge in Git?

Answer:

A merge where the branch pointer moves forward without creating a new merge commit because there are no divergent commits.

66. How do you initialize a Git repository?

Answer:

git init initializes a new Git repository in the current directory.

67. How do you clone an existing repository?

Answer:

git clone <repository-url> copies a remote repository to your local machine.

68. How do you check the status of a Git repository?

Answer:

git status shows staged, unstaged, and untracked files.

69. How do you configure your Git username and email?

Answer:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

70. What is the purpose of .gitignore?

Answer:

It specifies files or directories for Git to ignore, preventing unnecessary files (e.g., logs, build files) from being tracked.

71. How do you add changes to the staging area?**Answer:**

git add <file> or git add . to stage all changes.

72. How do you commit changes in Git?**Answer:**

git commit -m "Commit message" creates a snapshot of staged changes.

73. How do you view commit history?**Answer:**

git log shows commit history; git log --oneline gives a condensed view.

74. How do you revert a commit?**Answer:**

- git revert <commit-hash>: Creates a new commit that undoes changes.
 - git reset --hard <commit-hash>: Resets branch to a previous commit (destructive).
-

75. How do you delete a file from Git repository?**Answer:**

git rm <file> removes a file and stages the deletion for commit.

76. What is GitHub used for in software development?**Answer:**

Hosting Git repositories, version control collaboration, code review, issue tracking, CI/CD integration, and project management.

77. What are GitHub forks and pull requests?**Answer:**

- **Fork:** A copy of a repository under your GitHub account to experiment independently.

- **Pull Request (PR):** Proposes changes from one branch/fork to be merged into another repository.
-

78. How do you make a GitHub repository private?

Answer:

During repository creation, select **Private** or change visibility in repository **Settings** → **General** → **Change repository visibility**.

79. How do you connect a local Git repository to GitHub?

Answer:

```
git remote add origin <repository-url>
git push -u origin main
```

80. What is SSH vs HTTPS for GitHub authentication?

Answer:

- **SSH:** Uses keys for secure authentication without passwords.
 - **HTTPS:** Uses username and password/token for authentication. SSH is preferred for secure automation.
-

81. What is Docker and why is it used?

Answer:

Docker is a containerization platform that packages applications and their dependencies into isolated containers, ensuring consistent environments across development, testing, and production.

82. What is the difference between a container and a virtual machine?

Answer:

- **Container:** Lightweight, shares host OS kernel, faster startup, smaller footprint.
 - **VM:** Full OS, larger footprint, slower startup, runs on hypervisor.
-

83. What are the main components of Docker?

Answer:

- **Docker Engine:** Core runtime for building and running containers
- **Docker Images:** Templates for containers

- **Docker Containers:** Running instances of images
 - **Docker Hub:** Repository for sharing images
-

84. What is a Dockerfile?

Answer:

A text file with instructions to build a Docker image, including base image, dependencies, commands, and exposed ports.

85. What are the advantages of using Docker?

Answer:

- Consistency across environments
 - Lightweight and fast deployment
 - Isolation of dependencies
 - Simplified CI/CD workflows
 - Easy scaling of applications
-

86. How do you list all running Docker containers?

Answer:

`docker ps`

87. How do you list all containers, including stopped ones?

Answer:

`docker ps -a`

88. How do you remove a container?

Answer:

`docker rm <container_id_or_name>`

89. How do you stop a running container?

Answer:

`docker stop <container_id_or_name>`

90. How do you view container logs?

Answer:

`docker logs <container_id_or_name>`

91. What is docker run used for?

Answer:

To create and start a new container from a Docker image.

92. How do you run a container in detached mode?

Answer:

`docker run -d <image_name>`

93. How do you map ports between host and container?

Answer:

`docker run -p <host_port>:<container_port> <image_name>`

94. How do you mount a volume into a container?

Answer:

`docker run -v <host_path>:<container_path> <image_name>`

95. How do you pass environment variables to a container?

Answer:

`docker run -e VAR_NAME=value <image_name>`

96. How do you list all Docker images?

Answer:

`docker images`

97. How do you build a Docker image from a Dockerfile?

Answer:

`docker build -t <image_name>:<tag> .`

98. How do you remove a Docker image?**Answer:**

```
docker rmi <image_name_or_id>
```

99. What is Docker Compose?**Answer:**

A tool for defining and running multi-container Docker applications using a YAML file (docker-compose.yml) to configure services, networks, and volumes.

100. How do you start services defined in a docker-compose.yml file?**Answer:**

```
docker-compose up
```

101. What is a Docker Registry?**Answer:**

A Docker Registry is a storage and distribution system for Docker images. Public registries (like Docker Hub) or private registries allow developers to share and pull images.

102. What is the difference between Docker Hub and a private registry?**Answer:**

- **Docker Hub:** Public cloud registry for sharing images globally.
 - **Private Registry:** Self-hosted or restricted registry for secure, internal image sharing.
-

103. How do you push an image to Docker Hub?**Answer:**

```
docker login  
docker tag <local_image> <username>/<repo>:<tag>  
docker push <username>/<repo>:<tag>
```

104. How do you pull an image from a Docker Registry?**Answer:**

```
docker pull <image_name>:<tag>
```

105. What is the difference between docker pull and docker fetch?

Answer:

There is no docker fetch command in standard Docker; docker pull is used to download images from a registry.

106. How do you list images in a Docker Registry?

Answer:

For Docker Hub, use the web UI; for private registries, use API calls or CLI tools depending on the registry setup.

107. What is Docker Engine?

Answer:

Docker Engine is the core component of Docker that enables building, running, and managing containers on a host machine.

108. What are the main components of Docker Engine?

Answer:

- **Docker Daemon (dockerd)** – runs as a background service
 - **Docker CLI (docker)** – command-line interface to interact with the daemon
 - **REST API** – for programmatic communication
-

109. How do you check the Docker Engine version?

Answer:

docker version

110. How do you start, stop, and restart the Docker Engine?

Answer:

On Linux:

```
sudo systemctl start docker
sudo systemctl stop docker
sudo systemctl restart docker
```

111. What types of storage does Docker support?

Answer:

- **Volumes:** Managed by Docker, persistent, ideal for databases
 - **Bind Mounts:** Maps host directory to container
 - **Tmpfs:** Temporary in-memory storage
-

112. How do you create a Docker volume?

Answer:

`docker volume create <volume_name>`

113. How do you list Docker volumes?

Answer:

`docker volume ls`

114. How do you inspect a Docker volume?

Answer:

`docker volume inspect <volume_name>`

115. How do you remove a Docker volume?

Answer:

`docker volume rm <volume_name>`

116. What is the difference between a volume and a bind mount?

Answer:

- **Volume:** Managed by Docker, independent of host directory structure
 - **Bind Mount:** Uses existing host directory; container accesses exact host files
-

Docker Networking

117. What are the types of Docker networks?

Answer:

- **Bridge:** Default network for standalone containers
- **Host:** Shares host network stack

- **Overlay:** Enables communication across multiple Docker hosts (Swarm)
 - **None:** No network access
-

118. How do you list Docker networks?

Answer:

`docker network ls`

119. How do you create a custom Docker network?

Answer:

`docker network create <network_name>`

120. How do containers communicate within a Docker network?

Answer:

Containers on the same network can reach each other using container names as hostnames; Docker manages DNS resolution automatically.